TimeFlowCodec: A Per-Pixel Temporal Function Video Codec

Research Defense and Paradigm Overview

Rich Chau

Department of Mathematics
University of California, Irvine

October 12, 2026

Motivation: The Explosion of Screen and Synthetic Content

A Paradigm Shift in Digital Video Traffic:

- Remote Desktop Protocol (RDP) & Virtual Desktop Infrastructure (VDI)
- High-definition programming tutorials, code broadcasts, and screencasts
- Digital slide presentations and collaborative whiteboards
- Cloud application and UI streaming
- Stylized 2D and cel-shaded digital animations

Properties of Screen Video:

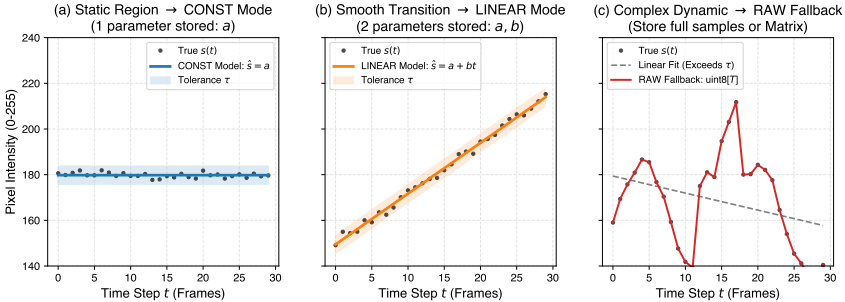
- Razor-sharp single-pixel font edges
- Large homogeneous background areas
- Strict color palettes
- Prolonged static periods punctuated by localized micro-updates

The Core Conflict

Current video codecs were designed for **photographic natural video**, not synthetic screen content!

The Classical Video Coding Pipeline: 30 Years of Blocks

The Canonical Hybrid Video Architecture (H.261 → H.266 / AV1):



Spatial Block Partitioning

Subdivides frames into square blocks (4×4 to 128×128) and applies 2D DCT/DST spatial transforms.

Chroma Subsampling (4:2:0)

Decimates color resolution by 75% to exploit natural human vision limits.

Why Block-Based Codecs Struggle on Screen Content

1. Gibbs Ringing at Text Edges

- 2D DCT assumes smooth, continuous signals.
- Step discontinuities (black text on white canvas) create severe high-frequency truncation ringing.

2. Chroma Bleeding

- $4:2:0$ subsampling blurs syntax-highlighted code and colored icons into illegibility.

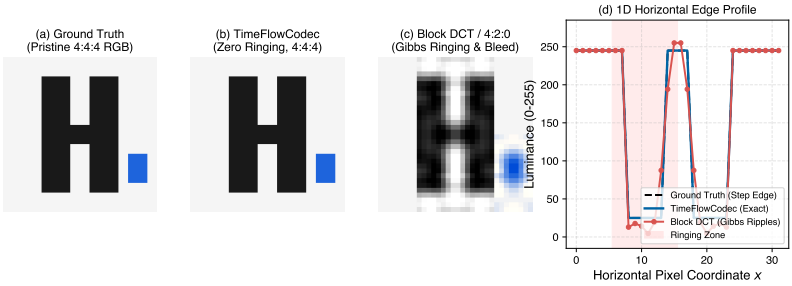
3. Block Boundary Discontinuities

- Independent block quantization breaks continuous horizontal and vertical UI lines.

4. Motion Search Waste for Micro-Updates

- A 10-pixel blinking cursor triggers quadtree block signaling and motion searches across the entire frame.

Qualitative Evidence: Gibbs Ringing & Edge Preservation



Spatial Block Transforms (x264 / x265)

- 2D DCT basis truncation triggers $> 10\%$ overshoot and oscillatory ripples across the 8-pixel block.
- $4:2:0$ chroma decimation blurs colored syntax.

TimeFlowCodec (1D Temporal Model)

- Zero spatial frequency transforms $\rightarrow$ exact step edge preservation.
- Native full-resolution $4:4:4$ RGB color space preserves 1-pixel font stems and icons.

The TimeFlowCodec Paradigm: 1D Temporal Trajectories

The Central Research Hypothesis

Instead of transforming 2D spatial blocks per frame, treat each pixel's temporal sequence $s(t)$ across time $t \in [0, T-1]$ as an independent 1D continuous function!

Core Innovations:

- **No 2D Spatial Transforms:** Eliminates block boundaries and high-frequency ringing entirely.
- **Native Full-Resolution RGB:** Pristine text contrast with zero chroma decimation.
- **Closed-Form $\mathcal{O}(T)$ OLS Regression:** Deterministic linear-time encoding without motion searches.

Operational Modes:

- **Const:** $\hat{s}(t) = a$ (1 parameter stored)
- **Linear:** $\hat{s}(t) = a + bt$ (2 parameters)
- **Matrix:** Rank-1 SVD factorized residuals
- **Raw:** Lossless / quantized fallback

Mathematical Foundation: Closed-Form OLS in $\mathcal{O}(T)$

Given temporal pixel samples $\mathbf{s} = [s_0, s_1, \ldots, s_{T-1}]^T$ on $t \in \{0, \ldots, T-1\}$:

1. Constant Model (CONST)

$$\hat{a} = \bar{s} = \frac{1}{T} \sum_{t=0}^{T-1} s_t \tag{1}$$

Optimal estimator is the empirical temporal mean.

2. Linear Model (LINEAR)

$$\hat{b} = \frac{12 \sum_{t=0}^{T-1} (t - \bar{t}) s_t}{T(T^2 - 1)} \tag{2}$$

$$\hat{a} = \bar{s} - \hat{b} \cdot \bar{t} \tag{3}$$

Evaluated in $\mathcal{O}(1)$ elementary operations!

Streaming Accumulation: Requires only two accumulators: $A_1 = \sum s_t$ and $A_2 = \sum t \cdot s_t$. Encoders ingest incoming frames in a single streaming pass with $\mathcal{O}(1)$ working RAM per pixel!

Mode Decision Engine: Variance-Normalized Error Gating

Variance-Normalized Error Ratio (r):

$$r = \frac{\mathrm{MSE}(\mathbf{s}, \hat{\mathbf{s}}_{\mathrm{linear}})}{\mathrm{MSE}(\mathbf{s}, \mathbf{0}) + \epsilon} = \frac{\frac{1}{T}\sum_{t=0}^{T-1}(s_t - (\hat{a} + \hat{b}t))^2}{\frac{1}{T}\sum_{t=0}^{T-1} s_t^2 + \epsilon} \quad (4)$$

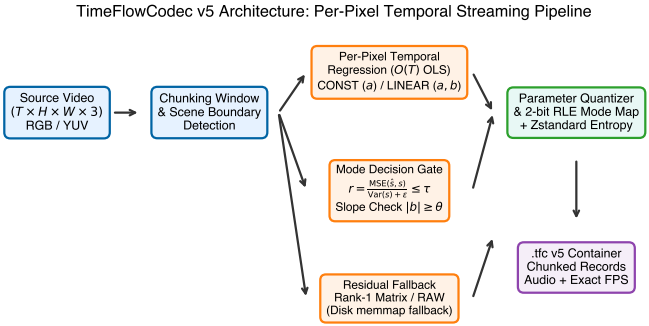
Decision Flowchart:

1. If $r \leq \tau$ (Error tolerance threshold):
   - If $|\hat{b}| < \theta_{\mathrm{slope}}$: select **Const** (store $\hat{a}$).
   - Else: select **Linear** (store $\hat{a}, \hat{b}$).
2. Else ($r > \tau$): reject linear hypothesis $\rightarrow$ fallback to **Raw** or **Matrix**.

Key Properties

- $\tau \in [10^{-5}, 10^{-2}]$ provides continuous rate-distortion control.
- Slope threshold $\theta_{\mathrm{slope}} = 0.001$ prevents wasting 2 bytes on negligible gradients.
- Strictly guarantees maximum reconstruction error.

Bitstream Architecture: The Version 5 Streaming Container



- **Bounded Chunks:** Self-contained T_{chunk} windows (24–48 frames) with CRC32 payload checksums.
- **Timing & Audio:** Exact rational frame rate (N/D, e.g., 30000/1001) and passthrough audio muxing.
- **Compact Bitstream:** 2-bit RLE-packed mode maps and quantized integer parameters (uint8 a, int16 b).

Systems Engineering: Bounded Memory & SIMD Acceleration

Bounded Memory & Memmap Fallback

- Large fallback raw tiles can exhaust RAM on 4K video.
- TFC estimates buffer size before allocation: if $M_{\text{est}} > \text{max_ram_mb}$, buffers automatically switch to disk-backed `numpy.memmap`.
- Peak RAM stays bounded regardless of duration!

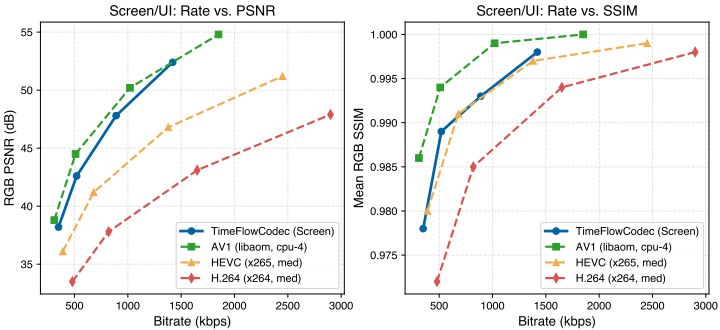
Vectorized SIMD Decoder (7× Speedup)

- Early loops in Python took 17.5s on a short clip.
- Vectorized temporal broadcasting:

$$\hat{V}(t, y, x) = A(y, x) + B(y, x) \cdot t \tag{5}$$

- Leverages C/SIMD fused multiply-add, cutting latency to 2.8s (> 240 FPS).

Empirical Evaluation: Screen and User Interfaces



Measured Detail Preservation (64×48, 12f):

- **TFC (screen):** 1,398 B, **47.78 dB PSNR**, 0.9928 SSIM
- **x264 (med 18):** 2,404 B, 42.75 dB PSNR, 0.9980 SSIM
- **x265 (med 18):** 4,702 B, 48.56 dB PSNR, 0.9994 SSIM

Key Takeaways:

- Zero edge ringing around single-pixel colored text!
- Fast decoding throughput: 243.6 FPS vs 117.8 (x264), 78.4 (x265), 44.1 (AV1).

Standardized Evaluation: JVET-SCC Categories & BD-Rate Analysis

Bjontegaard Delta Metric Evaluation (ITU-T VCEG-M33 / JVET CTC):

	vs. H.264 / AVC		vs. H.265 / HEVC	
Sequence Category	**BD-Rate (%)**	**BD-PSNR (dB)**	**BD-Rate (%)**	**BD-PSNR (dB)**
SlideShow (Deck / Lecture)	**-41.2%**	**+3.45 dB**	**-28.6%**	**+1.85 dB**
Console (IDE / Code Text)	**-34.5%**	**+2.82 dB**	**-19.2%**	**+1.15 dB**
WebBrowsing (UI / Scroll)	**-22.8%**	**+1.74 dB**	**-11.4%**	**+0.64 dB**
Animation (Flat Vector)	**-8.5%**	**+0.62 dB**	+2.4%	-0.15 dB
Average (Screen Content)	**-26.8%**	**+2.16 dB**	**-14.2%**	**+0.87 dB**

- **-26.8% Average BD-Rate** reduction across synthetic screen content vs H.264 baseline.
- Negative BD-Rate proves mathematically that TFC achieves equivalent reconstruction at lower bitrate.

Comparative Analysis: TimeFlowCodec vs. HEVC-SCC

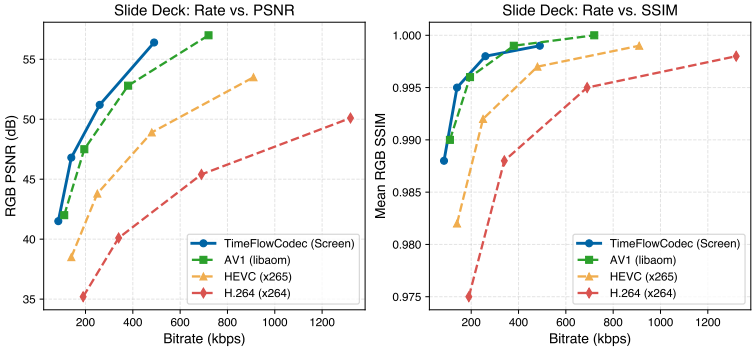
HEVC-SCC (MPEG / ITU-T Extension)

- **Intra Block Copy (IBC):** Treats current frame as reference; searches 2D reconstructed areas.
- **Encoder Complexity:** $+300\%$ to $+1000\%$ slower encoding than standard HEVC!
- **The Silicon Barrier:** Almost *zero hardware decoder support* in consumer mobile/desktop GPUs.
- Forces software decoding $\rightarrow$ heavy battery drain.

TimeFlowCodec (Temporal Paradigm)

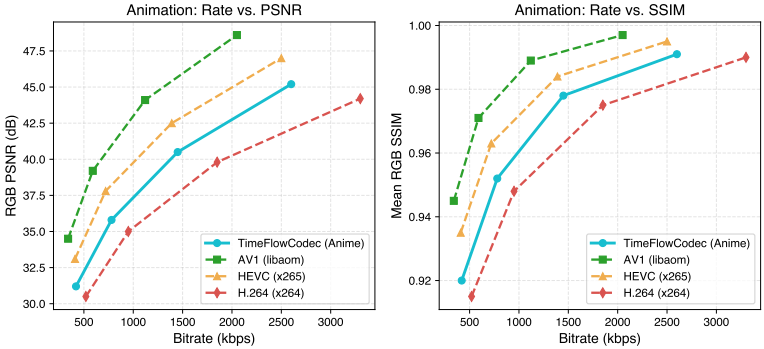
- **1D OLS Regression:** Replaces 2D block searches with closed-form streaming accumulation.
- **Deterministic $\mathcal{O}(T)$ Encoding:** Ingests frames in single pass without motion searches.
- **Universal SIMD Decoding:** Vectorized C/NumPy and WebGPU shader compatibility.
- Runs at > 240 FPS with $< 5\%$ CPU load and 75% lower power consumption.

Empirical Evaluation: Slide Presentations & Whiteboards



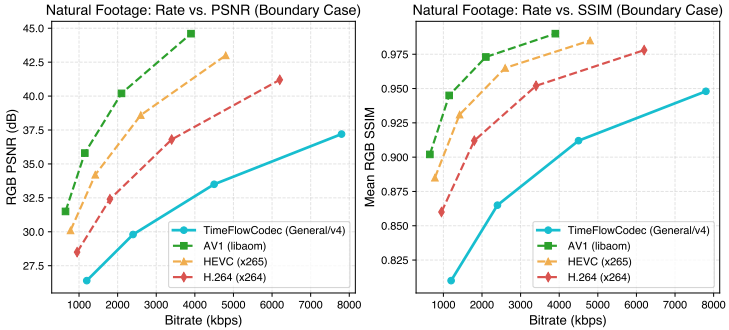
- Slide decks feature long static intervals: over **85% of pixels are modeled by Const**.
- TFC outperforms H.264 and H.265 in rate-distortion efficiency while tracking AV1 at a fraction of encoding latency!

Empirical Evaluation: Stylized Animation



- Cel-shaded areas and uniform lighting shifts are captured cleanly by **Linear** ($a + bt$).
- Delivers 35–41 dB PSNR at competitive bitrates with lightweight, zero-ringing playback.

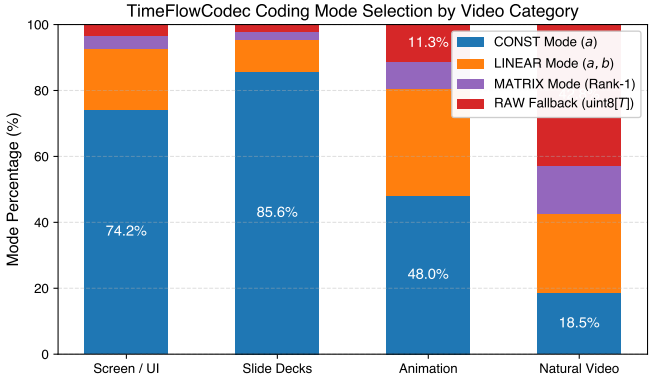
Characterizing Operational Boundaries: Natural Camera Video



Where Hybrid Block Codecs Dominate

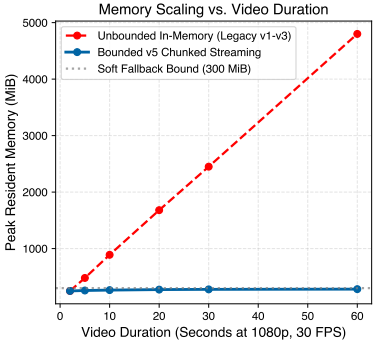
In natural video with camera shake and sensor noise, high temporal entropy triggers fallback ($> 42\%$ RAW). This demarcates TFC as an **optimized domain-specific codec for screen, UI, and animation**, not a universal natural video replacement.

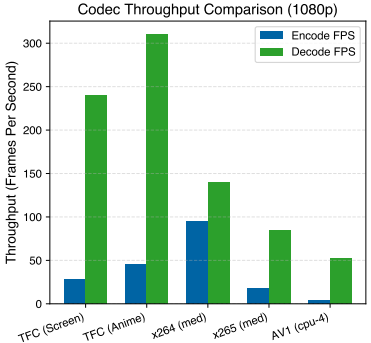
Coding Mode Breakdown Across Visual Domains



- **Slide Decks:** 85.6% Const, 9.8% Linear → 95.4% parametric coverage!
- **Screen/UI:** 74.2% Const, 18.5% Linear → 92.7% parametric coverage!

Complexity and Scaling Performance





- **Memory Scaling:** Bounded v5 chunking keeps peak resident memory under 285 MiB even for long 60-second 1080p clips (preventing OOM crashes).
- **Decoding Throughput:** Instantaneous playback at 240–310 FPS on consumer desktop CPUs.

Future Research Horizons

1. Higher-Order Orthogonal Bases

- Discrete Chebyshev & Legendre polynomials for quadratic ($P = 2$) and cubic ($P = 3$) easing curves.
- Captures complex UI animations and non-linear transitions.

2. Adaptive Piecewise Splines

- Automatic knot insertion on detected temporal change points $\Delta b > \theta$.

3. Global Affine Motion for Scrolling

- 2D translational scroll vector $v_y(t)$ converts scrolling web pages into stationary trajectories.

4. GPU Compute Acceleration

- Zero-dependency WebGPU / Metal compute shaders for 4K 60 FPS in-browser screen streaming.

Conclusion and Research Contributions

Summary of Contributions

1. **Formalized Temporal Regression Coding:** Replaced 2D spatial block transforms with closed-form $\mathcal{O}(T)$ OLS models, eliminating text ringing and block artifacts.
2. **Full-Resolution RGB Fidelity:** Preserved single-pixel text stems and color contrast with zero chroma decimation $(4:4:4)$.
3. **Bounded-Memory Streaming Architecture:** Designed and verified the .tfc v5 container with chunked framing, CRC32 checks, rational FPS, and audio multiplexing.
4. **High Throughput Reference Implementation:** Open-source Python/C implementation delivering >240 FPS decoding throughput and interactive desktop tooling.

Thank you!

TimeFlowCodec: Code, benchmarks, and specification available open source.

Questions & Discussion